

CS5344 Course Project: Tabular Data Augmentation for Anomaly Detection

1 Problem Formulation

Description. We study **tabular data augmentation for anomaly detection** on two datasets, and participants are required to complete the augmentation task on both datasets. For each dataset, participants are given a labeled training set containing both normal and abnormal samples and should apply a generation/augmentation method to produce a fixed number of normal-only synthetic samples. The objective is to maximize the **utility** of the synthetic data, measured by the anomaly-detection performance achieved when predefined downstream detectors are trained on the submitted synthetic samples using a fixed training scheme and evaluated on a fixed private test set. Thus, participants are encouraged to design generation/augmentation methods that produce synthetic normal data that is particularly useful for improving downstream anomaly detection performance, rather than merely reproducing the observed training distribution.

Data. Let

$$\mathbf{D} = \{\mathbf{x}_i\}_{i=1}^n$$

denote a tabular dataset containing n samples. Each sample consists of m feature columns,

$$\mathbf{x}_i = (x_{i1}, \dots, x_{im}),$$

where each column may represent either a numerical feature (e.g., 5.86) or a categorical feature (e.g., 0, 1, 2). Each sample $\mathbf{x}_i$ is associated with a binary anomaly label $y_i \in \{0, 1\}$, where $y_i = 0$ indicates a normal sample and $y_i = 1$ indicates an abnormal sample. Let

$$\mathbf{y} = \{y_i\}_{i=1}^n$$

denote the corresponding label vector. We refer to $(\mathbf{D}, \mathbf{y})$ as the labeled dataset. The labeled dataset is partitioned by rows into mutually disjoint training, validation, and test sets:

$$(\mathbf{D}_{\text{train}}, \mathbf{y}_{\text{train}}), \quad (\mathbf{D}_{\text{valid}}, \mathbf{y}_{\text{valid}}), \quad (\mathbf{D}_{\text{test}}, \mathbf{y}_{\text{test}}).$$

Only the labeled training set is released to participants during the competition. The validation and test sets remain private and contain both normal and abnormal samples. Their abnormal samples may include anomaly types observed in the training set as well as previously unseen anomaly types.

Synthetic Data Generation and Submission.

Participants design a generation/augmentation method using the released labeled training set to produce a synthetic dataset

$$\tilde{\mathbf{D}} = \{\tilde{\mathbf{x}}_i\}_{i=1}^N,$$

where N is the fixed submission size. The submitted dataset $\tilde{\mathbf{D}}$ must consist exclusively of samples intended to represent the normal data distribution. Participants may use both the feature values and anomaly labels in the released training set $(\mathbf{D}_{\text{train}}, \mathbf{y}_{\text{train}})$ when designing the generation/augmentation method. However, the final submission contains only the generated feature vectors and does not include anomaly labels.

Downstream Evaluation. Each submitted synthetic dataset $\tilde{\mathbf{D}}$ is used to train a pre-defined set of anomaly detection algorithms under a fixed training scheme specified by the organizers. The trained detectors assign anomaly scores to samples in the private validation and test sets, where higher scores indicate greater abnormality.

Performance is evaluated by comparing the anomaly scores with the corresponding ground-truth labels. We use threshold-free anomaly detection metric, **Area Under the Precision–Recall Curve (AUPRC)**.

Validation performance is reported on the public Kaggle leaderboard to support method development, while test-set performance remains hidden and is used for final evaluation.

2 Datasets

The competition uses two datasets, each independently instantiating the labeled dataset $(\mathbf{D}, \mathbf{y})$ and the training/validation/test partition defined in Section 1.

For each dataset, abnormal samples belong to a specific *anomaly type* (e.g., an attack category). We partition the set of anomaly types into three disjoint groups, $\mathcal{A}_{\text{seen}}$, $\mathcal{A}_{\text{val}}$, and $\mathcal{A}_{\text{test}}$, and construct the splits so that

$$\begin{aligned}\{\text{types of abnormal samples in } \mathbf{D}_{\text{train}}\} &= \mathcal{A}_{\text{seen}}, \\ \{\text{types of abnormal samples in } \mathbf{D}_{\text{valid}}\} &= \mathcal{A}_{\text{seen}} \cup \mathcal{A}_{\text{val}}, \\ \{\text{types of abnormal samples in } \mathbf{D}_{\text{test}}\} &= \mathcal{A}_{\text{seen}} \cup \mathcal{A}_{\text{val}} \cup \mathcal{A}_{\text{test}}.\end{aligned}$$

That is, the training set contains only anomaly types that participants can observe in advance; the validation set additionally contains anomaly types never seen during training; and the test set further contains at least one anomaly type never seen during either training or validation. This design lets us separately measure performance on seen anomaly types and on genuinely novel anomaly types, rather than relying on a single held-out type. The anomaly-type label itself (`attack_cat` / `class`, depending on the dataset) is used only internally by the organizers to construct and evaluate the splits; it is not released to participants and is not part of $\mathbf{x}_i$.

Dataset 1: NSL-KDD.

Description. NSL-KDD [2] is a refined version of the KDD Cup 1999 network intrusion dataset. Each row of the dataset records a single network connection, and its features describe measurable properties of that connection (e.g., duration, byte counts, protocol). Every

connection is labeled as either normal or abnormal: an abnormal connection corresponds to a network attack, and attacks span a number of distinct attack types (e.g., a SYN-flood denial-of-service attack and a network probing/surveillance attack).

Features. Each sample has $m = 41$ features: 38 numerical and 3 categorical (`service`, `protocol_type`, `flag`); no text/identifier columns are retained. Descriptions below follow the original KDD Cup 1999 feature definitions [2], grouped into *basic* features of a connection, *content* features flagging suspicious behaviour within a connection, and *traffic* features computed over a 2-second window (same host/service) or the preceding 100 connections (`dst_host_*`).

Feature	Description
<i>Basic features</i>	
<code>duration</code>	Length (seconds) of the connection
<code>protocol_type</code>	Protocol used in the connection (e.g., <code>tcp</code> , <code>udp</code> , <code>icmp</code>)
<code>service</code>	Destination network service (e.g., <code>http</code> , <code>telnet</code>)
<code>src.bytes</code>	Number of data bytes from source to destination
<code>dst.bytes</code>	Number of data bytes from destination to source
<code>flag</code>	Normal or error status of the connection
<code>land</code>	1 if connection is from/to the same host/port, 0 otherwise
<code>wrong_fragment</code>	Number of “wrong” fragments
<code>urgent</code>	Number of urgent packets
<i>Content features</i>	
<code>hot</code>	Number of “hot” indicators
<code>num_failed_logins</code>	Number of failed login attempts
<code>logged_in</code>	1 if successfully logged in, 0 otherwise
<code>num_compromised</code>	Number of “compromised” conditions
<code>root_shell</code>	1 if a root shell is obtained, 0 otherwise
<code>su_attempted</code>	1 if the <code>su root</code> command was attempted, 0 otherwise
<code>num_root</code>	Number of “root” accesses
<code>num_file_creations</code>	Number of file-creation operations
<code>num_shells</code>	Number of shell prompts
<code>num_access_files</code>	Number of operations on access-control files
<code>num_outbound_cmds</code>	Number of outbound commands in an FTP session
<code>is_host_login</code>	1 if the login belongs to the “host” list, 0 otherwise
<code>is_guest_login</code>	1 if the login is a “guest” login, 0 otherwise
<i>Traffic features (2-second window, same host/service)</i>	
<code>count</code>	Number of connections to the same host as the current one, in the past 2s
<code>srv_count</code>	Number of connections to the same service as the current one, in the past 2s
<code>serror_rate</code>	Percentage of <code>count</code> connections with a “SYN” error
<code>srv_serror_rate</code>	Percentage of <code>srv_count</code> connections with a “SYN” error
<code>rerror_rate</code>	Percentage of <code>count</code> connections with a “REJ” error
<code>srv_rerror_rate</code>	Percentage of <code>srv_count</code> connections with a “REJ” error

Feature	Description
<code>same_srv_rate</code>	Percentage of connections to the same service
<code>diff_srv_rate</code>	Percentage of connections to different services
<code>srv_diff_host_rate</code>	Percentage of connections to different hosts
<i>Traffic features (preceding 100 connections, same destination host)</i>	
<code>dst_host_count</code>	Number of the preceding 100 connections with the same destination host
<code>dst_host_srv_count</code>	Number of the preceding 100 connections with the same destination port
<code>dst_host_same_srv_rate</code>	Percentage of <code>dst_host_count</code> connections to the same service
<code>dst_host_diff_srv_rate</code>	Percentage of <code>dst_host_count</code> connections to different services
<code>dst_host_same_src_port_rate</code>	Percentage of <code>dst_host_srv_count</code> connections with the same source port
<code>dst_host_srv_diff_host_rate</code>	Percentage of <code>dst_host_srv_count</code> connections from different source hosts
<code>dst_host_serror_rate</code>	Percentage of <code>dst_host_count</code> connections with a “SYN” error
<code>dst_host_srv_serror_rate</code>	Percentage of <code>dst_host_srv_count</code> connections with a “SYN” error
<code>dst_host_rerror_rate</code>	Percentage of <code>dst_host_count</code> connections with a “REJ” error
<code>dst_host_srv_rerror_rate</code>	Percentage of <code>dst_host_srv_count</code> connections with a “REJ” error

Data Format. Only `train.csv` (features $x_{i1}, \dots, x_{i,41}$ and label y_i) is released to participants. `valid.csv` and `test.csv` remain private.

Dataset 2: UNSW-NB15.

Description. UNSW-NB15 [1] is a network traffic dataset generated with the IXIA PerfectStorm tool at the Cyber Range Lab of UNSW Canberra. Each row records a single network flow, and its features describe measurable properties of that flow (e.g., packet counts, TCP window size, connection state). Every flow is labeled as either normal or abnormal: an abnormal flow corresponds to a network attack, and attacks span several attack categories (e.g., a block-cipher attack applicable regardless of the underlying algorithm and denial-of-service traffic).

Features. Each sample has $m = 42$ features: 39 numerical and 3 categorical (`proto`, `service`, `state`); no text/identifier columns are retained. Descriptions below follow the official `UNSW-NB15_features.csv` released with the dataset [1].

Feature	Description
<code>proto</code>	Transaction protocol
<code>service</code>	Destination service, e.g. <code>http</code> , <code>ftp</code> , <code>smtp</code> , <code>ssh</code> , <code>dns</code> (- if not a well-known service)
<code>state</code>	Connection state and its dependent protocol (e.g., <code>FIN</code> , <code>CON</code> , <code>INT</code> , <code>REQ</code> , <code>RST</code>)
<code>dur</code>	Record total duration
<code>sbytes</code>	Source-to-destination transaction bytes

Feature	Description
dbytes	Destination-to-source transaction bytes
rate	Overall packet rate of the connection (added in the released train/test split; not part of the original 49-feature schema)
sttl	Source-to-destination time-to-live value
dttl	Destination-to-source time-to-live value
sloss	Source packets retransmitted or dropped
dloss	Destination packets retransmitted or dropped
sload	Source bits per second
dload	Destination bits per second
spkts	Source-to-destination packet count
dpkts	Destination-to-source packet count
swin	Source TCP window advertisement value
dwin	Destination TCP window advertisement value
stcpb	Source TCP base sequence number
dtcpb	Destination TCP base sequence number
smean	Mean flow packet size transmitted by the source
dmean	Mean flow packet size transmitted by the destination
trans_depth	Pipelined depth into the connection of an HTTP request/response transaction
response_body_len	Uncompressed content size transferred from the server's HTTP service
sjit	Source jitter (ms)
djit	Destination jitter (ms)
sinpkt	Source inter-packet arrival time (ms)
dinpkt	Destination inter-packet arrival time (ms)
tcprtt	TCP connection setup round-trip time (synack + ackdat)
synack	TCP setup time between the SYN and SYN_ACK packets
ackdat	TCP setup time between the SYN_ACK and ACK packets
is_sm_ips_ports	1 if source/destination IP and port numbers are both equal, 0 otherwise
ct_state_ttl	Number of records for each state, by source/destination TTL range
ct_flw_http_mthd	Number of flows with HTTP methods such as GET/POST
is_ftp_login	1 if the FTP session is accessed with a username/password, 0 otherwise
ct_ftp_cmd	Number of flows with a command in the FTP session
ct_srv_src	Number of connections with the same service and source address (last 100 connections)
ct_srv_dst	Number of connections with the same service and destination address (last 100 connections)
ct_dst_ltm	Number of connections with the same destination address (last 100 connections)
ct_src_ltm	Number of connections with the same source address (last 100 connections)
ct_src_dport_ltm	Number of connections with the same source address and destination port (last 100 connections)

Feature	Description
<code>ct_dst_sport_ltm</code>	Number of connections with the same destination address and source port (last 100 connections)
<code>ct_dst_src_ltm</code>	Number of connections with the same source and destination address (last 100 connections)

Data Format. As with Dataset 1, only `train.csv` is released; `valid.csv` and `test.csv` remain private for breakdown evaluation.

References

- [1] Nour Moustafa and Jill Slay. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *2015 Military Communications and Information Systems Conference (MilCIS)*, pages 1–6. IEEE, 2015.
- [2] Mahbod Tavallaei, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. A detailed analysis of the KDD CUP 99 data set. In *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*, pages 1–6. IEEE, 2009.